

Multi-Agent Insurance Claims Processing System

A complete LangGraph pipeline with **6 specialised AI agents** that process insurance claims through intake, policy validation, fraud detection, routing, decision-making, and customer communication.

Tech stack: LangGraph + Grok (xAI) via `langchain-xai`, with full deterministic fallbacks for offline operation.

Architecture Overview

```
START -> IntakeAgent -> PolicyValidationAgent ->
FraudDetectionAgent -> [Router]
    -> DecisionAgent (low risk)          -> CommunicationAgent -> END
    -> SeniorReviewAgent (escalated)    -> CommunicationAgent -> END
```

Each agent reads from and writes to a shared `ClaimState` (TypedDict), and appends to a `trace` log for full auditability.

1. Setup & Imports

```
In [1]: from __future__ import annotations

import json
import os
import re
from datetime import datetime
from typing import Any, Optional

try:
    from dotenv import load_dotenv
    load_dotenv()
except ImportError:
    pass

from langgraph.graph import StateGraph, END
from typing_extensions import TypedDict

api_key = os.environ.get('XAI_API_KEY')
print(f"XAI_API_KEY: {'SET' if api_key else 'NOT SET (using fallbacks)'}")
```

XAI_API_KEY: NOT SET (using fallbacks)

2. Mock Policy Database

Five sample policies with varying status, coverage type, and limits:

```
In [2]: MOCK_POLICIES = {
    "POL-10234": {
        "holder": "Jane Doe",
        "status": "active",
        "coverage_type": "auto",
        "coverage_limit": 10000,
        "deductible": 500,
    },
    "POL-20456": {
        "holder": "John Smith",
        "status": "active",
        "coverage_type": "home",
        "coverage_limit": 50000,
        "deductible": 1000,
    },
    "POL-30789": {
        "holder": "Alice Johnson",
        "status": "lapsed",
        "coverage_type": "auto",
        "coverage_limit": 15000,
        "deductible": 750,
    },
    "POL-40100": {
        "holder": "Bob Williams",
        "status": "active",
        "coverage_type": "health",
        "coverage_limit": 25000,
        "deductible": 200,
    },
    "POL-50321": {
        "holder": "Clara Davis",
        "status": "active",
        "coverage_type": "auto",
        "coverage_limit": 5000,
        "deductible": 300,
    },
}

print(f"Loaded {len(MOCK_POLICIES)} policies")
for pid, pdata in MOCK_POLICIES.items():
    print(f"  {pid}: {pdata['holder']} | {pdata['status']} | {pdata['coverage_type']}
```

Loaded 5 policies

```
POL-10234: Jane Doe | active | auto | $10,000
POL-20456: John Smith | active | home | $50,000
POL-30789: Alice Johnson | lapsed | auto | $15,000
POL-40100: Bob Williams | active | health | $25,000
POL-50321: Clara Davis | active | auto | $5,000
```

3. Shared State Schema

All agents read from and write to this `ClaimState` TypedDict:

```
In [3]: class ClaimState(TypedDict, total=False):
        # Raw input
        raw_text: str

        # IntakeAgent output
        claimant_name: str
        policy_number: str
        claim_type: str
        incident_date: str
        claim_amount: float
        description: str

        # PolicyValidationAgent output
        policy_found: bool
        policy_status: str
        coverage_valid: bool
        coverage_limit: float
        policy_notes: str

        # FraudDetectionAgent output
        fraud_score: int
        red_flags: list[str]
        fraud_reasoning: str

        # Routing result
        route: str

        # Decision output
        decision_status: str
        decision_reason: str

        # Communication output
        communication_letter: str

        # Trace Log
        trace: list[str]

        print('ClaimState schema defined')
```

ClaimState schema defined

4. LLM Helper (Grok via xAI)

Uses `ChatXAI` when `XAI_API_KEY` is set. Returns `None` on failure so agents can fall back to deterministic logic.

```
In [4]: def _get_llm():
        """Return a ChatXAI instance if XAI_API_KEY is available, else None."""
```

```

api_key = os.environ.get("XAI_API_KEY")
if not api_key:
    return None
try:
    from langchain_xai import ChatXAI
    return ChatXAI(
        model="grok-3-fast",
        temperature=0,
        xai_api_key=api_key,
    )
except Exception:
    return None

def _llm_call(system_prompt: str, user_prompt: str) -> Optional[str]:
    """Attempt an LLM call; return response text or None on any failure."""
    llm = _get_llm()
    if llm is None:
        return None
    try:
        from langchain_core.messages import SystemMessage, HumanMessage
        msgs = [SystemMessage(content=system_prompt), HumanMessage(content=user_pro
        resp = llm.invoke(msgs)
        return resp.content
    except Exception:
        return None

print('LLM helper ready')

```

LLM helper ready

5. Agent 1: IntakeAgent

Extracts structured fields (name, policy, type, date, amount, description) from free-form claim text. Uses LLM with JSON-only system prompt, with a regex-based fallback.

```

In [5]: def _fallback_intake(raw_text: str) -> dict[str, Any]:
    """Deterministic regex-based extraction from raw claim text."""
    text = raw_text

    name_match = re.search(
        r"[Cc]laimant\s*[:;-]\s*([A-Z][a-z]+(?:\s+[A-Z][a-z]+)*)", text
    )
    claimant_name = name_match.group(1).strip() if name_match else "Unknown"

    pol_match = re.search(r"(POL-\d{4,6})", text, re.IGNORECASE)
    policy_number = pol_match.group(1).upper() if pol_match else "UNKNOWN"

    claim_type = "auto"
    lower = text.lower()
    if "home" in lower or "property" in lower or "house" in lower:
        claim_type = "home"
    elif "health" in lower or "medical" in lower or "hospital" in lower:
        claim_type = "health"

```

```

date_match = re.search(
    r"(\d{4}-\d{2}-\d{2}|\d{1,2}/\d{1,2}/\d{2,4})", text
)
incident_date = date_match.group(1) if date_match else "Unknown"

amount_match = re.search(r"\$\s?([\d,]+(?:\.\d{2})?)", text)
claim_amount = 0.0
if amount_match:
    claim_amount = float(amount_match.group(1).replace(",", ""))

desc_match = re.search(
    r"(?:description|details|incident|happened|damage)[;:\s-]*(.+)",
    text,
    re.IGNORECASE | re.DOTALL,
)
if desc_match:
    description = desc_match.group(1).strip()[:300]
else:
    sentences = [s.strip() for s in re.split(r'[.!?]+', text) if s.strip()]
    description = sentences[-1] if sentences else text[:300]

return {
    "claimant_name": claimant_name,
    "policy_number": policy_number,
    "claim_type": claim_type,
    "incident_date": incident_date,
    "claim_amount": claim_amount,
    "description": description,
}

def intake_agent(state: ClaimState) -> dict:
    """Extract structured fields from raw claim text."""
    raw = state["raw_text"]
    trace = list(state.get("trace", []))
    trace.append(f"[IntakeAgent] Processing raw claim ({len(raw)} chars)")

    system_prompt = (
        "You are an insurance claim intake specialist. Extract the following fields "
        "from the raw claim text and return ONLY a valid JSON object with these key "
        "claimant_name, policy_number, claim_type (one of: auto, home, health), "
        "incident_date, claim_amount (number), description. "
        "Do NOT include any text outside the JSON."
    )

    result = None
    llm_response = _llm_call(system_prompt, raw)
    if llm_response:
        try:
            cleaned = re.sub(r"```(?:json)?\s*", "", llm_response).strip().rstrip(" ")
            result = json.loads(cleaned)
            trace.append("[IntakeAgent] LLM extraction successful")
        except (json.JSONDecodeError, KeyError):
            trace.append("[IntakeAgent] LLM response parse failed, using fallback")
            result = None

```

```

if result is None:
    result = _fallback_intake(raw)
    trace.append("[IntakeAgent] Used rule-based fallback extraction")

try:
    result["claim_amount"] = float(result["claim_amount"])
except (ValueError, TypeError):
    result["claim_amount"] = 0.0

trace.append(f"[IntakeAgent] Extracted: name={result.get('claimant_name')}, "
            f"policy={result.get('policy_number')}, type={result.get('claim_ty"
            f"amount=${result.get('claim_amount')}}")

return {**result, 'trace': trace}

print('IntakeAgent defined')

```

IntakeAgent defined

6. Agent 2: PolicyValidationAgent

Looks up the policy number in the mock database and checks:

- Policy exists
- Status is active
- Coverage type matches claim type
- Claim amount vs. coverage limit

```

In [6]: def policy_validation_agent(state: ClaimState) -> dict:
        """Validate claim against mock policy database."""
        trace = list(state.get("trace", []))
        policy_number = state.get("policy_number", "UNKNOWN")
        claim_type = state.get("claim_type", "")
        claim_amount = state.get("claim_amount", 0.0)

        trace.append(f"[PolicyValidationAgent] Looking up policy {policy_number}")

        policy = MOCK_POLICIES.get(policy_number)

        if policy is None:
            trace.append(f"[PolicyValidationAgent] Policy {policy_number} NOT FOUND")
            return {
                "policy_found": False,
                "policy_status": "not_found",
                "coverage_valid": False,
                "coverage_limit": 0.0,
                "policy_notes": f"Policy {policy_number} does not exist in our records.",
                "trace": trace,
            }

        status = policy["status"]
        cov_type = policy["coverage_type"]
        cov_limit = policy["coverage_limit"]

```

```

coverage_valid = (status == "active") and (cov_type == claim_type)

notes_parts = []
if status != "active":
    notes_parts.append(f"Policy is {status} (not active).")
if cov_type != claim_type:
    notes_parts.append(
        f"Coverage type mismatch: policy covers '{cov_type}' but claim is '{cla
    )
if claim_amount > cov_limit:
    notes_parts.append(
        f"Claim amount ${claim_amount:,.2f} exceeds coverage limit ${cov_limit:
    )
if not notes_parts:
    notes_parts.append("Policy is valid and covers this claim type.")

notes = " ".join(notes_parts)
trace.append(f"[PolicyValidationAgent] status={status}, coverage_valid={coverag
        f"limit=${cov_limit:,.2f}")

return {
    "policy_found": True,
    "policy_status": status,
    "coverage_valid": coverage_valid,
    "coverage_limit": float(cov_limit),
    "policy_notes": notes,
    "trace": trace,
}

print('PolicyValidationAgent defined')

```

PolicyValidationAgent defined

7. Agent 3: FraudDetectionAgent

Scores fraud risk (0-100) using either LLM analysis or a weighted heuristic fallback. Flags include: invalid policy, amount near/over limit, urgency language, missing dates, short descriptions.

```

In [7]: def _fallback_fraud(state: ClaimState) -> dict[str, Any]:
        """Deterministic heuristic fraud scoring."""
        score = 0
        flags = []

        if not state.get("coverage_valid", False):
            score += 30
            if state.get("policy_status") == "lapsed":
                flags.append("Policy is lapsed/inactive")
            elif not state.get("policy_found", False):
                flags.append("Policy not found in records")
            else:
                flags.append("Coverage type mismatch")

        limit = state.get("coverage_limit", 0)

```

```

amount = state.get("claim_amount", 0)
if limit > 0:
    ratio = amount / limit
    if ratio > 1.0:
        score += 25
        flags.append("Claim amount exceeds coverage limit")
    elif ratio > 0.85:
        score += 15
        flags.append("Claim amount very close to coverage limit")

raw = state.get("raw_text", "").lower()
urgency_words = ["urgent", "immediately", "asap", "right away", "emergency",
                 "rush", "desperate", "need money", "as soon as possible"]
for word in urgency_words:
    if word in raw:
        score += 10
        flags.append(f"Urgency language detected: '{word}'")
        break

if state.get("incident_date", "Unknown") == "Unknown":
    score += 15
    flags.append("Incident date missing or unclear")

desc = state.get("description", "")
if len(desc) < 20:
    score += 10
    flags.append("Very short or missing incident description")

if amount > 20000:
    score += 10
    flags.append(f"High claim amount (${amount:,.2f})")

score = min(score, 100)

reasoning = "Heuristic analysis: "
if flags:
    reasoning += "; ".join(flags) + "."
else:
    reasoning += "No significant risk indicators detected."

return {"fraud_score": score, "red_flags": flags, "fraud_reasoning": reasoning}

def fraud_detection_agent(state: ClaimState) -> dict:
    """Assess fraud risk using LLM or heuristic fallback."""
    trace = list(state.get("trace", []))
    trace.append("[FraudDetectionAgent] Analysing claim for fraud indicators")

    system_prompt = (
        "You are an insurance fraud detection specialist. Analyse the claim data be"
        "and return ONLY a JSON object with these keys:\n"
        "- fraud_score: integer 0-100 (0=no risk, 100=certain fraud)\n"
        "- red_flags: list of short strings describing risk indicators\n"
        "- fraud_reasoning: 1-2 sentence explanation\n\n"
        "Look for: policy invalid/lapsed, amount near or exceeding coverage limit,"
        "urgency or pressure language, missing documentation, unclear dates, "

```

```

        "inconsistencies between description and claim details."
    )

    user_prompt = (
        f"Claim data:\n"
        f"- Claimant: {state.get('claimant_name')}\n"
        f"- Policy: {state.get('policy_number')}\n"
        f"- Claim type: {state.get('claim_type')}\n"
        f"- Incident date: {state.get('incident_date')}\n"
        f"- Claim amount: ${state.get('claim_amount', 0):,.2f}\n"
        f"- Policy found: {state.get('policy_found')}\n"
        f"- Policy status: {state.get('policy_status')}\n"
        f"- Coverage valid: {state.get('coverage_valid')}\n"
        f"- Coverage limit: ${state.get('coverage_limit', 0):,.2f}\n"
        f"- Policy notes: {state.get('policy_notes')}\n\n"
        f"Raw claim text:\n{state.get('raw_text', '')}"
    )

    result = None
    llm_response = _llm_call(system_prompt, user_prompt)
    if llm_response:
        try:
            cleaned = re.sub(r"```(?:json)?\s*", "", llm_response).strip().rstrip("
            result = json.loads(cleaned)
            result["fraud_score"] = int(result["fraud_score"])
            result["red_flags"] = list(result["red_flags"])
            trace.append(f"[FraudDetectionAgent] LLM scoring: {result['fraud_score']}
        except (json.JSONDecodeError, KeyError, ValueError):
            trace.append("[FraudDetectionAgent] LLM parse failed, using fallback")
            result = None

    if result is None:
        result = _fallback_fraud(state)
        trace.append(f"[FraudDetectionAgent] Heuristic scoring: {result['fraud_score']}

    trace.append(f"[FraudDetectionAgent] Score={result['fraud_score']}, "
                f"Flags={result['red_flags']}")

    return {
        "fraud_score": result["fraud_score"],
        "red_flags": result["red_flags"],
        "fraud_reasoning": result["fraud_reasoning"],
        "trace": trace,
    }

print('FraudDetectionAgent defined')

```

FraudDetectionAgent defined

8. Conditional Router

Not an agent - a pure routing function. Escalates if:

- `fraud_score >= 60`
- Policy is invalid / not found

- `claim_amount > coverage_limit`

```
In [8]: def route_claim(state: ClaimState) -> str:
        """Route to DecisionAgent or SeniorReviewAgent based on risk factors."""
        fraud_score = state.get("fraud_score", 0)
        coverage_valid = state.get("coverage_valid", True)
        claim_amount = state.get("claim_amount", 0)
        coverage_limit = state.get("coverage_limit", float("inf"))
        policy_found = state.get("policy_found", True)

        if fraud_score >= 60:
            return "senior_review"
        if not policy_found or not coverage_valid:
            return "senior_review"
        if claim_amount > coverage_limit:
            return "senior_review"
        return "auto_approve"

        print('Router defined')
```

Router defined

9. Agent 4: DecisionAgent

Auto-approves low-risk claims with a clear rationale.

```
In [9]: def decision_agent(state: ClaimState) -> dict:
        """Auto-approve low-risk claims."""
        trace = list(state.get("trace", []))
        trace.append("[DecisionAgent] Auto-approving claim")

        reason = (
            f"Claim approved. Policy {state.get('policy_number')} is "
            f"{state.get('policy_status', 'active')} with valid coverage. "
            f"Fraud score is low ({state.get('fraud_score', 0)}/100). "
            f"Claim amount ${state.get('claim_amount', 0):.2f} is within the "
            f"coverage limit of ${state.get('coverage_limit', 0):.2f}."
        )

        trace.append(f"[DecisionAgent] Decision: approved")

        return {
            "route": "auto_approve",
            "decision_status": "approved",
            "decision_reason": reason,
            "trace": trace,
        }

        print('DecisionAgent defined')
```

DecisionAgent defined

10. Agent 5: SeniorReviewAgent

Handles escalated claims, listing the exact trigger(s) that caused escalation.

```
In [10]: def senior_review_agent(state: ClaimState) -> dict:
          """Escalate high-risk claims for senior review."""
          trace = list(state.get("trace", []))
          trace.append("[SeniorReviewAgent] Reviewing escalated claim")

          triggers = []
          if state.get("fraud_score", 0) >= 60:
              triggers.append(f"High fraud score ({state.get('fraud_score')}/100)")
          if not state.get("policy_found", True):
              triggers.append("Policy not found in records")
          if not state.get("coverage_valid", True):
              triggers.append(f"Invalid coverage (policy status: {state.get('policy_status')})")
          if state.get("claim_amount", 0) > state.get("coverage_limit", float("inf")):
              triggers.append(
                  f"Claim amount ${state.get('claim_amount', 0):.2f} exceeds "
                  f"coverage limit ${state.get('coverage_limit', 0):.2f}"
              )
          if not triggers:
              triggers.append("Escalated for manual review")

          reason = (
              f"Claim escalated for senior review. Trigger(s): {'; '.join(triggers)}. "
              f"Policy: {state.get('policy_number')}, Fraud score: {state.get('fraud_score')}"
          )

          trace.append(f"[SeniorReviewAgent] Decision: escalated - triggers: {triggers}")

          return {
              "route": "escalate",
              "decision_status": "escalated",
              "decision_reason": reason,
              "trace": trace,
          }

          print('SeniorReviewAgent defined')
```

SeniorReviewAgent defined

11. Agent 6: CommunicationAgent

Drafts an empathetic customer letter with different tone/template based on the decision (approved / escalated / denied).

```
In [11]: LETTER_APPROVED = """Dear {name},

Re: Insurance Claim - Policy {policy}

We are pleased to inform you that your insurance claim has been reviewed and approved.

Decision: APPROVED
{reason}
"""
```

Your claim of \${amount} will be processed shortly, subject to your policy deductibl

If you have any questions, please don't hesitate to contact our claims department.

Warm regards,
Claims Processing Team""

LETTER_ESCALATED = ""Dear {name},

Re: Insurance Claim - Policy {policy}

Thank you for filing your insurance claim. After our initial review, your claim has

Decision: ESCALATED FOR REVIEW
{reason}

A senior claims specialist will be assigned to your case and will contact you withi

If you have any urgent questions, please call our priority claims line.

Sincerely,
Claims Processing Team""

LETTER_DENIED = ""Dear {name},

Re: Insurance Claim - Policy {policy}

Thank you for submitting your insurance claim. After careful review, we regret to i

Decision: DENIED
{reason}

You have the right to appeal this decision within 30 days. Please contact our claim

We understand this may be disappointing, and we are here to help you through the pr

Respectfully,
Claims Processing Team""

```
def _fallback_communication(state: ClaimState) -> str:
    name = state.get("claimant_name", "Valued Customer")
    policy = state.get("policy_number", "N/A")
    status = state.get("decision_status", "escalated")
    reason = state.get("decision_reason", "")
    amount = f"{state.get('claim_amount', 0):,.2f}"

    if status == "approved":
        template = LETTER_APPROVED
    elif status == "denied":
        template = LETTER_DENIED
    else:
        template = LETTER_ESCALATED

    return template.format(name=name, policy=policy, reason=reason, amount=amount)
```

```

def communication_agent(state: ClaimState) -> dict:
    """Draft a customer-facing letter about the claim decision."""
    trace = list(state.get("trace", []))
    status = state.get("decision_status", "escalated")
    trace.append(f"[CommunicationAgent] Drafting letter for '{status}' decision")

    system_prompt = (
        "You are an empathetic insurance communications specialist. "
        "Draft a short, professional letter to the claimant about their claim decis
        "Use a warm but professional tone. Include the claimant's name, policy numb
        "decision status, and reason. Return ONLY the letter text, no JSON wrapping
    )

    user_prompt = (
        f"Claimant: {state.get('claimant_name')}\n"
        f"Policy: {state.get('policy_number')}\n"
        f"Claim amount: ${state.get('claim_amount', 0):,.2f}\n"
        f"Decision: {status}\n"
        f"Reason: {state.get('decision_reason')}\n\n"
        f"Please draft the letter."
    )

    letter = None
    llm_response = _llm_call(system_prompt, user_prompt)
    if llm_response and len(llm_response.strip()) > 50:
        letter = llm_response.strip()
        trace.append("[CommunicationAgent] LLM-generated letter")

    if letter is None:
        letter = _fallback_communication(state)
        trace.append("[CommunicationAgent] Used template-based letter")

    trace.append(f"[CommunicationAgent] Letter drafted ({len(letter)} chars)")

    return {
        "communication_letter": letter,
        "trace": trace,
    }

print('CommunicationAgent defined')

```

CommunicationAgent defined

12. Graph Construction & Compilation

Build the LangGraph state graph connecting all agents.

```

In [12]: def build_graph() -> StateGraph:
    """Build and return the compiled LangGraph state graph."""
    graph = StateGraph(ClaimState)

    graph.add_node("intake", intake_agent)
    graph.add_node("policy_validation", policy_validation_agent)

```

```

graph.add_node("fraud_detection", fraud_detection_agent)
graph.add_node("decision", decision_agent)
graph.add_node("senior_review", senior_review_agent)
graph.add_node("communication", communication_agent)

graph.set_entry_point("intake")
graph.add_edge("intake", "policy_validation")
graph.add_edge("policy_validation", "fraud_detection")

graph.add_conditional_edges(
    "fraud_detection",
    route_claim,
    {
        "auto_approve": "decision",
        "senior_review": "senior_review",
    },
)

graph.add_edge("decision", "communication")
graph.add_edge("senior_review", "communication")
graph.add_edge("communication", END)

return graph

def process_claim(raw_text: str):
    """Process a raw claim text through the full agent pipeline."""
    app = build_graph().compile()
    initial_state = {
        "raw_text": raw_text,
        "trace": [f"[System] Claim received at {datetime.now().isoformat()}"],
    }
    return app.invoke(initial_state)

print('Graph builder and process_claim ready')

```

Graph builder and process_claim ready

13. Pipeline Visualization (Mermaid)

```

In [13]: app = build_graph().compile()
mermaid_str = app.get_graph().draw_mermaid()
print(mermaid_str)

```

```

---
config:
  flowchart:
    curve: linear
---
graph TD;
  __start__([<p>__start__</p>]):::first
  intake(intake)
  policy_validation(policy_validation)
  fraud_detection(fraud_detection)
  decision(decision)
  senior_review(senior_review)
  communication(communication)
  __end__([<p>__end__</p>]):::last
  __start__ --> intake;
  decision --> communication;
  fraud_detection -. &nbsp;auto_approve&nbsp; .-> decision;
  fraud_detection -.-> senior_review;
  intake --> policy_validation;
  policy_validation --> fraud_detection;
  senior_review --> communication;
  communication --> __end__;
  classDef default fill:#f2f0ff,line-height:1.2
  classDef first fill-opacity:0
  classDef last fill:#bfb6fc

```

14. Sample Claims

```

In [14]: SAMPLE_CLAIMS = {
  "approved": (
    "Claimant: Jane Doe. Policy: POL-10234. Auto claim. "
    "Incident date 2026-08-01. Amount: $4,500. "
    "My car was rear-ended at a stoplight on Highway 101. "
    "The other driver admitted fault. Police report #PR-2026-5567 filed. "
    "Damage to rear bumper, trunk, and tail lights."
  ),
  "fraud_escalated": (
    "Claimant: Alice Johnson. Policy: POL-30789. Auto claim. "
    "Incident date 2026-07-15. Amount: $14,000. "
    "I need this processed URGENTLY -- my car was totaled in a hit-and-run. "
    "No witnesses. I need the money as soon as possible. "
    "Please rush this claim immediately."
  ),
  "over_limit_escalated": (
    "Claimant: Clara Davis. Policy: POL-50321. Auto claim. "
    "Incident date 2026-06-20. Amount: $7,500. "
    "A tree fell on my car during a storm. "
    "Damage to roof, windshield, and hood. "
    "Fire department report available."
  ),
}

```

```
for name, text in SAMPLE_CLAIMS.items():
    print(f"\n{name.upper()}: {text[:80]}...")
```

APPROVED: Claimant: Jane Doe. Policy: POL-10234. Auto claim. Incident date 2026-08-01. Amount: 4,500.

FRAUD_ESCALATED: Claimant: Alice Johnson. Policy: POL-30789. Auto claim. Incident date 2026-07-15. Amount: 10,000.

OVER_LIMIT_ESCALATED: Claimant: Clara Davis. Policy: POL-50321. Auto claim. Incident date 2026-06-20. Amount: 15,000.

15. Demo Run 1: Approved Claim (Low Risk)

Jane Doe with active policy POL-10234, 4,500 auto claim well within 10,000 limit. Expected: **APPROVED** with low fraud score.

```
In [15]: result_approved = process_claim(SAMPLE_CLAIMS["approved"])

print('=' * 60)
print('SCENARIO: APPROVED CLAIM')
print('=' * 60)
print(f"Claimant:    {result_approved.get('claimant_name')}")
print(f"Policy:      {result_approved.get('policy_number')}")
print(f"Claim Type:   {result_approved.get('claim_type')}")
print(f"Amount:      ${result_approved.get('claim_amount', 0):.2f}")
print(f"Policy Valid: {result_approved.get('coverage_valid')}")
print(f"Fraud Score: {result_approved.get('fraud_score')}/100")
print(f"Red Flags:   {result_approved.get('red_flags')}")
print(f"Decision:    {result_approved.get('decision_status')}")
print(f"Reason:      {result_approved.get('decision_reason')}")
print(f"\nLetter:\n{result_approved.get('communication_letter')}")
print(f"\nTrace:")
for t in result_approved.get('trace', []):
    print(f"    {t}")
```

```
=====
SCENARIO: APPROVED CLAIM
=====
```

```
Claimant:    Jane Doe
Policy:      POL-10234
Claim Type:  auto
Amount:      $4,500.00
Policy Valid: True
Fraud Score: 0/100
Red Flags:   []
Decision:    approved
Reason:      Claim approved. Policy POL-10234 is active with valid coverage. Fraud score is low (0/100). Claim amount $4,500.00 is within the coverage limit of $10,000.00.
```

```
Letter:
Dear Jane Doe,
```

```
Re: Insurance Claim - Policy POL-10234
```

```
We are pleased to inform you that your insurance claim has been reviewed and approved.
```

```
Decision: APPROVED
Claim approved. Policy POL-10234 is active with valid coverage. Fraud score is low (0/100). Claim amount $4,500.00 is within the coverage limit of $10,000.00.
```

```
Your claim of $4,500.00 will be processed shortly, subject to your policy deductible. Please allow 5-7 business days for the payment to be issued.
```

```
If you have any questions, please don't hesitate to contact our claims department.
```

```
Warm regards,
Claims Processing Team
```

```
Trace:
[System] Claim received at 2026-09-05T13:43:41.064994
[IntakeAgent] Processing raw claim (260 chars)
[IntakeAgent] Used rule-based fallback extraction
[IntakeAgent] Extracted: name=Jane Doe, policy=POL-10234, type=auto, amount=$4500.00
[PolicyValidationAgent] Looking up policy POL-10234
[PolicyValidationAgent] status=active, coverage_valid=True, limit=$10,000.00
[FraudDetectionAgent] Analysing claim for fraud indicators
[FraudDetectionAgent] Heuristic scoring: 0
[FraudDetectionAgent] Score=0, Flags=[]
[DecisionAgent] Auto-approving claim
[DecisionAgent] Decision: approved
[CommunicationAgent] Drafting letter for 'approved' decision
[CommunicationAgent] Used template-based letter
[CommunicationAgent] Letter drafted (591 chars)
```

16. Demo Run 2: Fraud-Escalated Claim

Alice Johnson with **lapsed** policy POL-30789, \$14,000 auto claim with urgency language ("URGENTLY", "as soon as possible", "rush"). Expected: **ESCALATED** due to invalid policy (lapsed).

```
In [16]: result_fraud = process_claim(SAMPLE_CLAIMS["fraud_escalated"])

print('=' * 60)
print('SCENARIO: FRAUD ESCALATED')
print('=' * 60)
print(f"Claimant:    {result_fraud.get('claimant_name')}")
print(f"Policy:       {result_fraud.get('policy_number')}")
print(f"Claim Type:    {result_fraud.get('claim_type')}")
print(f"Amount:        ${result_fraud.get('claim_amount', 0):,.2f}")
print(f"Policy Valid: {result_fraud.get('coverage_valid')}")
print(f"Fraud Score: {result_fraud.get('fraud_score')}/100")
print(f"Red Flags:     {result_fraud.get('red_flags')}")
print(f"Decision:      {result_fraud.get('decision_status')}")
print(f"Reason:        {result_fraud.get('decision_reason')}")
print(f"\nLetter:\n{result_fraud.get('communication_letter')}")
print(f"\nTrace:")
for t in result_fraud.get('trace', []):
    print(f"    {t}")
```

=====

SCENARIO: FRAUD ESCALATED

=====

Claimant: Alice Johnson
Policy: POL-30789
Claim Type: auto
Amount: \$14,000.00
Policy Valid: False
Fraud Score: 55/100
Red Flags: ['Policy is lapsed/inactive', 'Claim amount very close to coverage limit', 'Urgency language detected: 'urgent']
Decision: escalated
Reason: Claim escalated for senior review. Trigger(s): Invalid coverage (policy status: lapsed). Policy: POL-30789, Fraud score: 55/100.

Letter:
Dear Alice Johnson,

Re: Insurance Claim - Policy POL-30789

Thank you for filing your insurance claim. After our initial review, your claim has been escalated to our senior review team for further evaluation.

Decision: ESCALATED FOR REVIEW
Claim escalated for senior review. Trigger(s): Invalid coverage (policy status: lapsed). Policy: POL-30789, Fraud score: 55/100.

A senior claims specialist will be assigned to your case and will contact you within 2-3 business days to discuss next steps. We appreciate your patience during this process.

If you have any urgent questions, please call our priority claims line.

Sincerely,
Claims Processing Team

Trace:
[System] Claim received at 2026-09-05T13:43:41.094599
[IntakeAgent] Processing raw claim (257 chars)
[IntakeAgent] Used rule-based fallback extraction
[IntakeAgent] Extracted: name=Alice Johnson, policy=POL-30789, type=auto, amount=\$14000.0
[PolicyValidationAgent] Looking up policy POL-30789
[PolicyValidationAgent] status=lapsed, coverage_valid=False, limit=\$15,000.00
[FraudDetectionAgent] Analysing claim for fraud indicators
[FraudDetectionAgent] Heuristic scoring: 55
[FraudDetectionAgent] Score=55, Flags=['Policy is lapsed/inactive', 'Claim amount very close to coverage limit', 'Urgency language detected: 'urgent']
[SeniorReviewAgent] Reviewing escalated claim
[SeniorReviewAgent] Decision: escalated - triggers: ['Invalid coverage (policy status: lapsed)']
[CommunicationAgent] Drafting letter for 'escalated' decision
[CommunicationAgent] Used template-based letter
[CommunicationAgent] Letter drafted (654 chars)

17. Demo Run 3: Over-Limit Escalated Claim

Clara Davis with active policy POL-50321 (5,000*limit*), *but* 7,500 claim amount exceeds the limit. Expected: **ESCALATED** due to over-limit trigger.

```
In [17]: result_over = process_claim(SAMPLE_CLAIMS["over_limit_escalated"])

print('=' * 60)
print('SCENARIO: OVER-LIMIT ESCALATED')
print('=' * 60)
print(f"Claimant:    {result_over.get('claimant_name')}")
print(f"Policy:      {result_over.get('policy_number')}")
print(f"Claim Type:    {result_over.get('claim_type')}")
print(f"Amount:       ${result_over.get('claim_amount', 0):,.2f}")
print(f"Policy Valid: {result_over.get('coverage_valid')}")
print(f"Coverage Limit: ${result_over.get('coverage_limit', 0):,.2f}")
print(f"Fraud Score: {result_over.get('fraud_score')}/100")
print(f"Red Flags:    {result_over.get('red_flags')}")
print(f"Decision:     {result_over.get('decision_status')}")
print(f"Reason:       {result_over.get('decision_reason')}")
print(f"\nLetter:\n{result_over.get('communication_letter')}")
print(f"\nTrace:")
for t in result_over.get('trace', []):
    print(f"    {t}")
```

=====

SCENARIO: OVER-LIMIT ESCALATED

=====

Claimant: Clara Davis
Policy: POL-50321
Claim Type: auto
Amount: \$7,500.00
Policy Valid: True
Coverage Limit: \$5,000.00
Fraud Score: 25/100
Red Flags: ['Claim amount exceeds coverage limit']
Decision: escalated
Reason: Claim escalated for senior review. Trigger(s): Claim amount \$7,500.00 exceeds coverage limit \$5,000.00. Policy: POL-50321, Fraud score: 25/100.

Letter:
Dear Clara Davis,

Re: Insurance Claim - Policy POL-50321

Thank you for filing your insurance claim. After our initial review, your claim has been escalated to our senior review team for further evaluation.

Decision: ESCALATED FOR REVIEW
Claim escalated for senior review. Trigger(s): Claim amount \$7,500.00 exceeds coverage limit \$5,000.00. Policy: POL-50321, Fraud score: 25/100.

A senior claims specialist will be assigned to your case and will contact you within 2-3 business days to discuss next steps. We appreciate your patience during this process.

If you have any urgent questions, please call our priority claims line.

Sincerely,
Claims Processing Team

Trace:
[System] Claim received at 2026-09-05T13:43:41.124352
[IntakeAgent] Processing raw claim (205 chars)
[IntakeAgent] Used rule-based fallback extraction
[IntakeAgent] Extracted: name=Clara Davis, policy=POL-50321, type=auto, amount=\$7500.00
[PolicyValidationAgent] Looking up policy POL-50321
[PolicyValidationAgent] status=active, coverage_valid=True, limit=\$5,000.00
[FraudDetectionAgent] Analysing claim for fraud indicators
[FraudDetectionAgent] Heuristic scoring: 25
[FraudDetectionAgent] Score=25, Flags=['Claim amount exceeds coverage limit']
[SeniorReviewAgent] Reviewing escalated claim
[SeniorReviewAgent] Decision: escalated - triggers: ['Claim amount \$7,500.00 exceeds coverage limit \$5,000.00']
[CommunicationAgent] Drafting letter for 'escalated' decision
[CommunicationAgent] Used template-based letter
[CommunicationAgent] Letter drafted (667 chars)

18. Summary

Scenario	Claimant	Decision	Key Trigger
Low Risk	Jane Doe	Approved	All clear
Fraud Risk	Alice Johnson	Escalated	Lapsed policy + urgency language
Over Limit	Clara Davis	Escalated	7,500 >5,000 limit

The pipeline correctly routes claims through the conditional branching logic, with full trace logs for auditability.